

Reflection

Data choices

I used `int` for the day, month, year, hour, and minute because those values are whole numbers. I used `double` for the sensor value because readings can have decimal places. I used `String` for sensor ID, sensor type, and zone because they are names or labels, not values that need maths done on them.

I made `Timestamp` its own class because the date and time belong together. It also means the formatting code for the timestamp is not mixed into `SensorReading`.

Class structure

I tried to keep each class focused on one job. `Main` starts everything. `Menu` deals with user choices. `GreenhouseDataStorage` loads and saves the CSV file. `SensorArray` stores the readings. `Math` does the statistics. `Validator` checks input.

At first I had more statistic methods inside `SensorArray`, but that made the class do too much. Moving the calculations into `Math` made the program easier to follow, because `SensorArray` is now mostly about storing and finding readings. `SensorReading` contains a `Timestamp`, which matches the way one reading in the CSV has one date and time attached to it.

Menu design

The menu uses a `while` loop and an `if / else if` chain. This is simple, but it is also easy to read. I kept the zones and sensor types hardcoded because the program only needs `ZoneA`, `ZoneB`, `ZoneC`, and the four sensor types from the data.

This also meant I could remove the older code that worked out distinct zones and sensor types from the CSV file. That code worked, but it was not really needed anymore.

When a reading is added or deleted, the program saves the whole array back to `data.csv`. This is needed because changing the array in memory does not update the file by itself.

Challenges

One challenge was keeping the validation consistent. The CSV loading code and the menu input code both need to check the same kind of values, so I moved those checks into `Validator`. This helped keep the rules in one place instead of spreading them across the program.

Another challenge was checking duplicate sensor IDs. I put that check in `SensorArray.hasSensorID()` because `SensorArray` stores all the readings, so it makes sense for it to search through them.

One issue was that changing menu options to hardcoded choices made some old code unnecessary. The distinct zone and distinct sensor type methods were removed because the menu no longer needs to build those lists from the CSV file.

Another issue was saving. I realised that adding a reading to `SensorArray` only changes the program while it is running. `saveCSV()` has to be called if the change should stay in `data.csv`.

Improvements

The date validation could be better. Right now it accepts 31 as a day for every month. A better version would check the actual month and reject dates like 31 April.

I would also add an edit option. At the moment, the user has to delete a reading and add it again if one field is wrong. With more time, I would also make the statistics output a bit neater by rounding long decimal values.